

1.7 Node.js and Jest: your first real business rule

i All the backend code for Encon is Node.js and all of it should have tests. Rather than writing a throwaway hello-world, I am going to write the single most important function in your project — the rule that decides whether a fridge reading is a problem.

It will also need to be tested (with jest) as stated above it is the single most important function.

Learning objectives

What are npm and `{package.json}` for?

Package.json is a special file that defines your node project. as you have done you had to change the script section of that file to allow your project to use JEST as the Js tester. so you have to define your node project from this file.

Npm is short for node package manager. It is the world largest software registry / library. It provides insane access to millions of pre-built open source code. It is useful to developer as they don't have to build everything from scratch (saves time).

What's the difference between a dependency and a dev dependency, and why does it matter?

DEF: A regular dependency is a package your app needs to run in production. A dev dependency is a tool you use only when writing or testing code.

Key Differences

- **Dependencies:** Required for runtime; included in the final deployed app.
- **Dev Dependencies:** Used only for building, testing, or formatting code; left out of production.
- **Examples (Dependencies):** React, Express, Axios.
- **Examples (Dev Dependencies):** Jest, ESLint, Prettier, TypeScript (compiler)

Why write unit tests at all — what do they buy you beyond "checking it works"?

Unit testing has a lot of benefits beyond 'just checking the work' They provide a reliable safety net for changes, serve as living documentation for how systems behave, and force developers to write cleaner, more modular code from the start. there are also many more benefit beside these such as;

- **Stops bugs early:** Catch errors right when you write the code.
- **Safe refactoring:** Change the internal code without fear of breaking old features.
- **Fast feedback:** Run tests in seconds instead of clicking through screens by hand

but these are some god examples.

What is a boundary condition, and why is it where bugs live?

A boundary condition is a specific value or state where a system's behavior changes, such as the edge of a range or the limit of a size. It is where bugs live because programmers often forget to handle these exact edge cases, leading to errors like off-by-one mistakes or crashes.

Task

 In your project repo, write one small pure function:

```
{{getReadingStatus(minTempC, maxTempC)}}
```

It returns `{{'excursion'}}` if `{{minTempC}}` is below 2 or `{{maxTempC}}` is above 8, and `{{'ok'}}` otherwise.

Then test it with Jest. Cover at least:

- A normal in-range reading — min 3, max 7 → ok
- Exactly on the boundaries — min 2, max 8 → ok
- Just outside — min 1.9 → excursion; max 8.1 → excursion
- Both out of range at once
- Rubbish input — no arguments, a string instead of a number, null.

Decide what should happen and test that it does

The order of working that I have laid out:

create a branch → check Node.js → initialise npm → install Jest → create the function → create the tests → run them → document your decisions → commit and push.

- ☒ create a branch
- ☒ check Node.js
- ☒ initialise npm
- ☒ install Jest
- ☒ create the function
- ☒ create the tests
- ☒ run them
- ☒ document your decision (throughout)

✓ commit and push (throughout)

Create a new branch

Because I am making the function that controls the temp, the new branch is called Fridge_TempFunc. This is where I will create the JS function for this section before merging it back with main.

NOTE: always branch off main. If you create a new branch while on a feature branch you will create a feature of a feature.

NOTE: when you want to get out of a directory in a terminal you use `cd ..`. The `..` takes you back one level from your current directory. Then you can use `cd` again to enter the desired directory, e.g. `cd encon-file`.

A more effective command is `cd ../encon-file`, which goes back one level with `..` and then into `encon-file` in a single command.

Check / install Node.js, Jest, npm

I have installed the package manager, JavaScript runtime, and testing environments on my PC and in the VS Code terminal. Hamza told me the setup is usually the worst, which it was — there were a lot of errors and the platforms either did not work or appeared to be downloaded but weren't. Now that this is all sorted we can begin the coding process.

NOTE: Hamza said to utilise AI whenever you get stuck. Just tell it your problem and it will help you.

Create the function

In my index.js file I inputted the code shown below:

```
1 /**
2  Checks whether a fridge temperature reading is within the safe range.
3
4  Safe range:
5  Minimum temperature must not be below 2°C.
6  Maximum temperature must not be above 8°C.
7
```

```

8  @param {number} minTempC - The lowest recorded temperature.
9  @param {number} maxTempC - The highest recorded temperature.
10 @returns {'ok' | 'excursion'} The status of the reading.
11
12 @throws {TypeError} If either input is missing or is not a valid
   number.
13 */
14
15 function getReadingStatus(minTempC, maxTempC) {
16   // Reject missing values, strings, null, NaN and Infinity.
17   if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {
18     throw new TypeError(
19       'minTempC and maxTempC must both be valid numbers'
20     );
21   }
22
23   // A reading is an excursion if either limit is outside the safe
   range.
24   if (minTempC < 2 || maxTempC > 8) {
25     return 'Excursion';
26   }
27   return 'Ok';
28
29 module.exports = { getReadingStatus };

```

What does it do / How does it work

This condition checks the **task rule**:

```
1 | if (minTempC < 2 || maxTempC > 8)
```

The `||` means **or**. Therefore, the function returns `'excursion'` when:

- the minimum temperature is below `2`, or
- the maximum temperature is above `8`.

Otherwise, it returns:

`'Ok'`

Exactly `2°C` and exactly `8°C` are acceptable because the code uses `<` and `>` rather than `<=` and `>=`.

Invalid-input decision

For invalid inputs, the function throws a `TypeError`.

For example, all of these will produce an error:

```

1 | getReadingStatus();
2 | getReadingStatus('3', 7);
3 | getReadingStatus(null, 7);

```

This is safer than returning `'ok'` or `'excursion'`. Invalid sensor data is different from a genuine temperature result and should not silently be treated as safe.

❌ **Error (problem that arose):** in my VS Code due to installing Node.js so I had a JavaScript runtime to work with. A folder called `node_modules` was created with over 5K files meaning if I were to commit the files it would also commit these 5K irrelevant folders to the repo.

To solve this issue I used a `.gitignore` file, which did:

The `.gitignore` file tells Git, "Ignore this folder and do not include it in commits," because it contains `node_modules/`. Git now ignores that large folder.

Create the tests

First I created a file called `index.test.js`.

📌 **NOTE:** An `index.test.js` file contains automated tests for a main `index.js` file or a primary folder. It uses tools like Jest (or the JavaScript testing framework that you have installed) to check that your JavaScript code works correctly and does not break.

Then I used the code shown below to create the tests.

```
1  const { getReadingStatus } = require('./index');
2
3  describe('getReadingStatus', () => {
4
5      test('returns ok for a normal in-range reading', () => {
6          expect(getReadingStatus(3, 7)).toBe('ok');
7      });
8
9      test('returns ok when exactly on the boundaries', () => {
10         expect(getReadingStatus(2, 8)).toBe('ok');
11     });
12
13     test('returns excursion when minimum temperature is just below 2',
14     () => {
15         expect(getReadingStatus(1.9, 7)).toBe('excursion');
16     });
17
18     test('returns excursion when maximum temperature is just above 8',
19     () => {
20         expect(getReadingStatus(3, 8.1)).toBe('excursion');
21     });
22
23     test('returns excursion when both temperatures are out of range', ()
24     => {
25         expect(getReadingStatus(1.9, 8.1)).toBe('excursion');
```

```

23 });
24
25 test('throws an error when no arguments are provided', () => {
26   expect(() => getReadingStatus()).toThrow(TypeError);
27 });
28
29 test('throws an error when a string is provided', () => {
30   expect(() => getReadingStatus('3', 7)).toThrow(TypeError);
31 });
32
33 test('throws an error when null is provided', () => {
34   expect(() => getReadingStatus(null, 7)).toThrow(TypeError);
35 });
36
37 });

```

These tests directly cover what the task is asking me to test. Below is the explanation of the different tests:

Test input	Expected result	What the test is checking	Why this test matters
<code>getReadingStatus(3, 7)</code>	<code>'ok'</code>	This checks a normal fridge reading where both the minimum and maximum temperatures are safely inside the allowed range of 2°C to 8°C. The function should reach the final <code>return 'ok';</code> statement because neither excursion condition is true.	This confirms that normal, valid fridge readings are not incorrectly marked as a problem.
<code>getReadingStatus(2, 8)</code>	<code>'ok'</code>	This checks the exact lower and upper boundaries. The condition in the function uses	Boundary values are common places for programming mistakes. If the code

		<code>minTempC < 2</code> and <code>maxTempC</code> <code>> 8</code> , so exactly 2°C and exactly 8°C should still be accepted.	accidentally used <code><= 2</code> or <code>>= 8</code> , this test would fail and expose the error.
<code>getReadingStatus(1.9, 7)</code>	'excursion'	This checks what happens when the minimum temperature is just below the safe limit. Because <code>1.9 < 2</code> is true, the first part of the condition causes the function to return <code>'excursion'</code> .	This proves that even a small amount below the minimum temperature is correctly identified as a temperature excursion.
<code>getReadingStatus(3, 8.1)</code>	'excursion'	This checks what happens when the maximum temperature is just above the safe upper limit. Because <code>8.1 > 8</code> is true, the function should return <code>'excursion'</code> .	This checks the upper boundary and confirms that even a small amount above 8°C is treated as a problem.
<code>getReadingStatus(1.9, 8.1)</code>	'excursion'	This checks a reading where both the minimum and	

		maximum temperatures are outside the safe range. Both sides of the condition are true, although only one needs to be true because the code uses the OR operator.	
<pre>getReadingStatus()</pre>	<pre>TypeError</pre>	This calls the function without supplying either argument. Both values become <code>undefined</code>. <code>Number.isFinite(undefined)</code> returns false, causing the validation section to throw a <code>TypeError</code>.	This ensures that missing sensor data cannot accidentally be treated as a valid fridge reading.
<pre>getReadingStatus('3', 7)</pre>	<pre>TypeError</pre>	This gives the function a string instead of a number. Even though <code>'3'</code> looks like a number, <code>Number.isFinite('3')</code> returns false because the	This makes the function strict about the type of data it receives and prevents incorrect or badly formatted data from being processed.

		value is still a string. The function therefore throws a <code>TypeError</code> .	
<code>getReadingStatus(null, 7)</code>	<code>TypeError</code>	This checks what happens when <code>null</code> is passed as the minimum temperature. <code>Number.isFinite(null)</code> returns false, so the validation code throws a <code>TypeError</code> .	This confirms that empty or missing values cannot accidentally be classified as a safe or unsafe fridge reading.

The tests also check two separate parts of my code. The first five tests check the actual **task rule**:

```

1 | if (minTempC < 2 || maxTempC > 8) {
2 |   return 'excursion';
3 | }
4 |
5 | return 'ok';

```

The final three tests check the **input validation**:

```

1 | if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {
2 |   throw new TypeError(
3 |     'minTempC and maxTempC must both be valid numbers'
4 |   );
5 | }

```

Running the tests / Result of the tests (were they successful or not)?

On running these tests all were supposed to pass. The expected result when running `npm test` should have been:

```
PASS ./index.test.js
```

Tests: 8 passed, 8 total

Test Suites: 1 passed, 1 total

❌ **Error:** Instead, the result on the CLI showed errors, as seen below:

```
Test Suites: 1 failed, 1 total
Tests:      5 failed, 3 passed, 8 total
Snapshots:  0 total
Time:       0.155 s
Ran all test suites.
umarhussain@Encons-MacBook-Air 4-week-project %
```

This error appeared because I had changed the expected outcomes of 'ok' and 'excursion' to 'Ok' and 'Excursion', hence causing these tests to fail due to the outcomes being different. The three that passed were the type-error tests, as they had the same expected outcome as the test results (this shows the importance of even one letter in code).

To fix this I changed the values back and re-ran the tests; the outcome then came back positive/passed.

```
getReadingStatus
✓ returns ok for a normal in-range reading (1 ms)
✓ returns ok when exactly on the boundaries
✓ returns excursion when minimum temperature is just below 2
✓ returns excursion when maximum temperature is just above 8
✓ returns excursion when both temperatures are out of range
✓ throws an error when no arguments are provided (2 ms)
✓ throws an error when a string is provided
✓ throws an error when null is provided (1 ms)

Test Suites: 1 passed, 1 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       0.124 s, estimated 1 s
Ran all test suites.
umarhussain@Encons-MacBook-Air 4-week-project %
```

📌 **NOTE:** Luke gave me a tip before if test your doing is case sensite.then uses .Lowercase() at the end of a string, to make are it always stays in the case you need it in. e.g

string thisisastring 'OK'

Return thisisastring.Lowcase()

would always make sure the ok is always returned in lowercase, no matter if the string inputted is in all upper case.

Commit and push & Documenting decisions

Committing and pushing were done throughout development and testing. Whenever I felt the part I was working on was finished, I committed it and then pushed it. Both tasks were completed throughout this sub-development of the project.

 Questions, queries and pushback

Task queries = I utilized AI to create the code. I understand what the code does and what the functionality is. I used AI for speed and because I believe most of Enocn's coding is done by AI, so being able to understand the code is sufficient. I will be using AI anyway when I start development. However, I'm not sure if this is a correct statement or not (not sure if I am allowed to do it)???

Learning objectives = I am unsure if they are the right answers as I have just googled them; I need to ask Luke or Hamza especial for the last question. (also note what they have said to you: *in the what I gathered from Luke was:*).